```
import random
import numpy as np

# ---------------------------------------------------------------
# 1. PROBLEM CONFIGURATION (Bhutanese Rural Healthcare Example)
# ---------------------------------------------------------------
# GPS/Map coordinates modeled as arbitrary grids (e.g., in a mountain valley)
VILLAGE_COORDINATES = np.array([
    [15.0, 80.0],  # Village A (Isolated mountain ridge)
    [30.0, 20.0],  # Village B (Deep valley settlement)
    [85.0, 50.0]   # Village C (Remote border community)
])

# Search space boundaries for the new ambulance hub [Min, Max] for X and Y coor
BOUNDS = np.array([[0.0, 100.0], [0.0, 100.0]])

def calculate_fitness(hub_position):
    """
    Fitness Function: Calculates the mean Euclidean distance from the proposed
    hub position to all three isolated villages.
    Objective: MINIMIZE this average travel distance/time.
    """
    distances = np.linalg.norm(VILLAGE_COORDINATES - hub_position, axis=1)
    return np.mean(distances)

# ---------------------------------------------------------------
# 2. PARTICLE CLASS DEFINITION
# ---------------------------------------------------------------
class Particle:
    def __init__(self, bounds):
        # Initialize random positions within the defined geographical space
        self.position = np.array([random.uniform(bounds[d][0], bounds[d][1]) fo

        # Initialize random initial velocities
        self.velocity = np.array([random.uniform(-1, 1) for _ in range(len(boun

        # Evaluate initial fitness
        self.fitness = calculate_fitness(self.position)

        # Initialize personal best (pbest) to the starting position
        self.pbest_position = np.copy(self.position)
        self.pbest_fitness = self.fitness

    def update_position(self, bounds):
        """Applies velocity to change position and clamps it within regional bo
        self.position += self.velocity

        # Boundary constraint handling (Clamping)
        for d in range(len(bounds)):
            if self.position[d] < bounds[d][0]:
                self.position[d] = bounds[d][0]
            elif self.position[d] > bounds[d][1]:
                self.position[d] = bounds[d][1]

# ---------------------------------------------------------------
# 3. PSO ALGORITHM CORE IMPLEMENTATION
# ---------------------------------------------------------------
def particle_swarm_optimization(swarm_size=30, max_iter=100, w=0.7, c1=1.5, c2=
```

```
    """
    Executes the PSO workflow.
    Parameters:
      w: Inertia weight (controls momentum)
      c1: Cognitive coefficient (pull towards personal best)
      c2: Social coefficient (pull towards swarm best)
    """
    # Step 1: Initialize Swarm
    swarm = [Particle(BOUNDS) for _ in range(swarm_size)]

    # Identify initial global best (gbest)
    gbest_particle = min(swarm, key=lambda p: p.pbest_fitness)
    gbest_position = np.copy(gbest_particle.pbest_position)
    gbest_fitness = gbest_particle.pbest_fitness

    # Step 2: Optimization Loop
    for iteration in range(max_iter):
        # Adaptive Inertia Weight Decay (transitions from exploration to explo
        current_w = w * (1.0 - (iteration / max_iter))

        for particle in swarm:
            # Generate stochastic random scaling factors
            r1 = random.random()
            r2 = random.random()

            # 2a. Calculate & Update Velocity
            cognitive_component = c1 * r1 * (particle.pbest_position - particle
            social_component = c2 * r2 * (gbest_position - particle.position)
            particle.velocity = (current_w * particle.velocity) + cognitive_com

            # 2b. Update Position
            particle.update_position(BOUNDS)

            # 2c. Evaluate New Fitness
            particle.fitness = calculate_fitness(particle.position)

            # 2d. Update Personal Best (pbest)
            if particle.fitness < particle.pbest_fitness:
                particle.pbest_fitness = particle.fitness
                particle.pbest_position = np.copy(particle.position)

                # 2e. Update Global Best (gbest)
                if particle.fitness < gbest_fitness:
                    gbest_fitness = particle.fitness
                    gbest_position = np.copy(particle.position)

        # Optional: Print tracking metrics for progress visual checking
        if (iteration + 1) % 20 == 0 or iteration == 0:
            print(f"Iteration {iteration+1}/{max_iter} -> Current Best Avg Dist

    return gbest_position, gbest_fitness

# ------------------------------------------------------------
# 4. EXECUTION
# ------------------------------------------------------------
if __name__ == "__main__":
    print("--- Starting Particle Swarm Optimization for Bhutan Transit Optimiza

    # Run the PSO function
    best_hub_coords, minimum_average_distance = particle_swarm_optimization(
```

```
            swarm_size=40,
            max_iter=100
        )

        print("\n--- Optimization Complete ---")
        print(f"Optimal Ambulance Hub Location Coordinates: X = {best_hub_coords[0]
        print(f"Minimized Average Distance to All Villages: {minimum_average_distan
```

```
--- Starting Particle Swarm Optimization for Bhutan Transit Optimization

Iteration 1/100 -> Current Best Avg Distance: 38.4885 km
Iteration 20/100 -> Current Best Avg Distance: 38.3988 km
Iteration 40/100 -> Current Best Avg Distance: 38.3988 km
Iteration 60/100 -> Current Best Avg Distance: 38.3988 km
Iteration 80/100 -> Current Best Avg Distance: 38.3988 km
Iteration 100/100 -> Current Best Avg Distance: 38.3988 km

--- Optimization Complete ---
Optimal Ambulance Hub Location Coordinates: X = 40.88, Y = 44.99
Minimized Average Distance to All Villages: 38.40 km
```